



Национальный Исследовательский
Университет «Высшая Школа Экономики»

2026

Линейная регрессия и градиентное обучение

Лекция 3



Содержание

- 1 Валидация и кросс-валидация
- 2 Регуляризация линейной модели
- 3 Градиентный спуск
- 4 SGD и mini-batch



Валидация и кросс-валидация

01

Разбиения train/test и train/validation/test
K-fold и стратификация
Проверка на будущих данных



Как можно проверять модель

Сколько раз проверяем

Train / test

Обучаем модель и проверяем
на отложенных данных.

Train / validation / test

Отдельно подбираем настройки
и оцениваем итог.

K-fold

Повторяем обучение и проверку
на разных блоках.

Как делим данные

Стратификация

Сохраняем доли классов.

Разбиение во времени

Учимся на прошлом, проверяем
будущее.

Группы

Связанные объекты держим вместе.
Подробно здесь не разбираем.



Обучение, валидация и тест

Валидация оценивает качество на данных, которые модель не использовала при обучении. По этой оценке выбираем настройки и сравниваем модели.

Часть данных	Для чего используем
Train	Находим веса модели
Validation	Выбираем модель и её настройки, например силу регуляризации
Test	Один раз оцениваем результат после завершения подбора



Стратификация

Стратификация делит данные так, чтобы в каждой части сохранились примерно те же доли классов, что и во всей выборке.



```
train_test_split(X, y, test_size=0.2, stratify=y)
```



K-fold: каждый блок побывает тестовым

Одно разбиение даёт одну оценку. K-fold повторяет проверку K раз: каждый раз обучаем новую модель, оставляя один блок для проверки.

Разбиение	Блок 1	Блок 2	Блок 3	Блок 4	Блок 5
1	Test	Train	Train	Train	Train
2	Train	Test	Train	Train	Train
3	Train	Train	Test	Train	Train
4	Train	Train	Train	Test	Train
5	Train	Train	Train	Train	Test



Результаты по фолдам

На каждом тестовом фолде считаем одну и ту же метрику.
Получаем несколько оценок, которые можно сравнить и обобщить.

Фолд	1	2	3	4	5
MSE	4	6	5	3	7

Среднее: 5

Медиана: 5

Ст. отклонение: 1,41

Можно смотреть минимум, максимум и разброс.

Так видно не только типичное качество, но и зависимость от разбиения.



StratifiedKFold: доли классов в каждом фолде

В выборке 9 объектов класса А и 3 класса Б: соотношение 3 к 1.

В каждом тестовом фолде сохраняем его: 3 объекта А и 1 объект Б.

Класс	А	А	Б	А	А	А	Б	А	А	А	Б	А
Фолд 1	Test	Train	Test	Train	Test	Train	Train	Train	Test	Train	Train	Train
Фолд 2	Train	Test	Train	Train	Train	Test	Test	Train	Train	Test	Train	Train
Фолд 3	Train	Train	Train	Test	Train	Train	Train	Test	Train	Train	Test	Test

Каждый объект один раз попадает в Test.

На остальных объектах заново обучаем модель.



Проверка прогноза на будущих данных

Для временных данных обучаемся на прошлом и проверяемся на будущем.
Случайное перемешивание нарушит этот порядок.

Разбиение	Янв.	Фев.	Март	Апр.
1	Train	Test		
2	Train	Train	Test	
3	Train	Train	Train	Test

Май оставляем для финальной проверки после подбора настроек.
В этой схеме он не участвует ни в одном разбиении.



Scaler обучаем на train, применяем ко всем частям

StandardScaler и MinMaxScaler запоминают параметры обучающих данных.
На validation и test используем те же параметры, без нового обучения.

Train: fit_transform

Находим параметры преобразования
и преобразуем обучающие данные.

```
scaler = StandardScaler()  
X_train = scaler.fit_transform(X_train)  
X_test = scaler.transform(X_test)
```

Validation и test: transform

Только применяем преобразование.
Параметры остаются прежними.



Регуляризация линейной модели

02

Штраф за большие веса
Ridge, Lasso и геометрия ограничений
Выбор силы регуляризации



Большой вес усиливает погрешность признака

Две модели при $x = 1$ дают один и тот же прогноз: 1.

При измерении ошиблись на 0,1 и передали модели $x = 1,1$.

Вес равен 10

$$a(x) = 10x - 9$$

Новый прогноз: $10 \cdot 1,1 - 9 = 2$.

Прогноз изменился на 1.

Вес равен 1

$$b(x) = x$$

Новый прогноз: 1,1.

Прогноз изменился только на 0,1.



Регуляризация: ошибка плюс штраф

Теперь хотим одновременно хорошо предсказывать ответы и не делать веса слишком большими.

$$J_{\lambda}(w) = Q(w) + \lambda R(w) \rightarrow \min_w$$

$Q(w)$: ошибка прогнозов

$R(w)$: штраф за величину весов

$\lambda \geq 0$ задаёт силу штрафа. При $\lambda = 0$ получаем обычный МНК.



Перед регуляризацией масштабируем признаки

Одинаковое расстояние можно записать в метрах или километрах.
Вес изменится в 1000 раз, хотя прогноз останется тем же.

Штраф зависит от численного значения веса. Поэтому признаки
приводим к сопоставимому масштабу до обучения модели.

StandardScaler

Вычитает среднее и делит
на стандартное отклонение.

MinMaxScaler

Приводит обучающие значения
к заданному диапазону, обычно $[0, 1]$.

Три вида регуляризации линейной модели

К одной и той же ошибке $Q(w)$ добавляем разные штрафы.

Ridge

$$Q(w) + \lambda \sum_{j=1}^d w_j^2$$

Lasso

$$Q(w) + \lambda \sum_{j=1}^d |w_j|$$

Elastic Net

$$Q(w) + \lambda_1 \sum_{j=1}^d |w_j| + \lambda_2 \sum_{j=1}^d w_j^2$$



Ridge: сумма квадратов весов

$$J_{\lambda}(w) = Q(w) + \lambda \sum_{j=1}^d w_j^2 \rightarrow \min_w$$

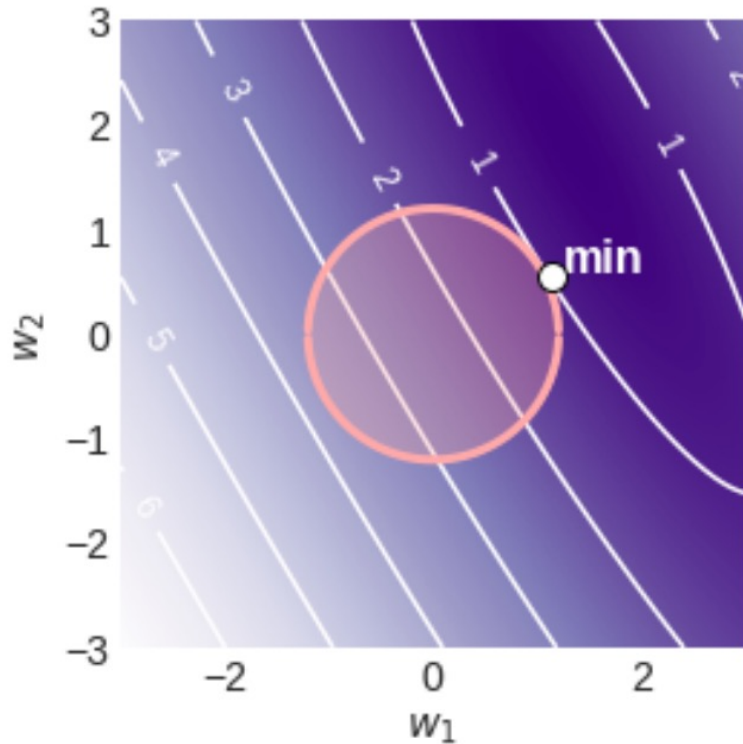
Ridge добавляет к ошибке сумму квадратов весов.

Чем больше λ , тем сильнее модель штрафует за большие веса.

Свободный член w_0 обычно не штрафует.

Он задаёт общий уровень ответа.

Геометрия Ridge: круг допустимых весов



Оси показывают два веса w_1 и w_2 .
Каждая кривая соединяет точки
с одинаковой ошибкой Q .

$$w_1^2 + w_2^2 \leq s$$

Ограничиваем суммарную величину весов.
Ищем наименьшую ошибку внутри круга.
Белая точка показывает решение.



У Ridge есть аналитическое решение

Для квадратичной ошибки веса Ridge можно найти из системы линейных уравнений, как и в обычном МНК.

$$(A^T A + \ell \lambda D) w = A^T y$$

$$w_{\lambda}^* = (A^T A + \ell \lambda D)^{-1} A^T y$$

$D = \text{diag}(0, 1, \dots, 1)$: свободный член не штрафует.

При $\lambda > 0$ решение существует и единственно даже при зависимых признаках.



Lasso: штраф за модули коэффициентов

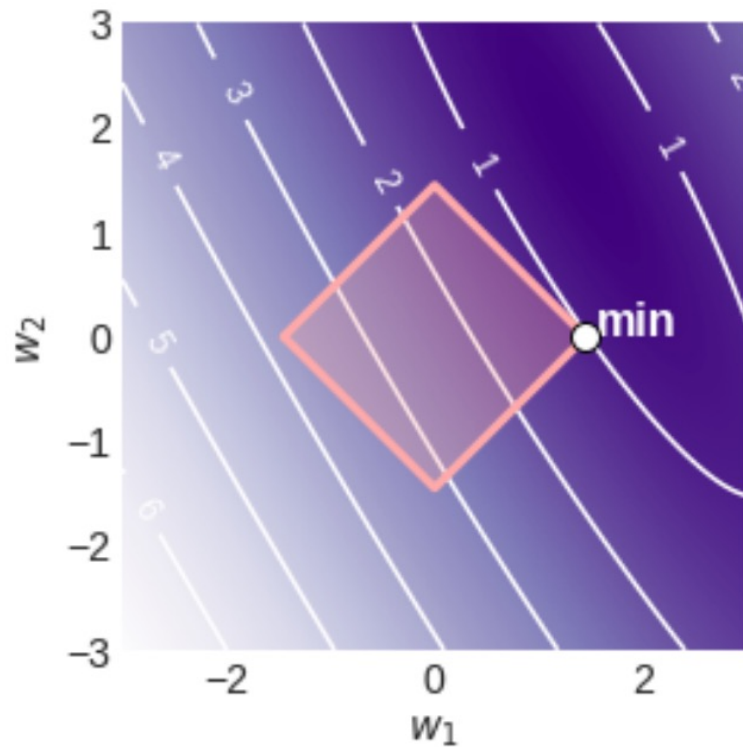
$$J_{\lambda}(w) = Q(w) + \lambda \sum_{j=1}^d |w_j| \rightarrow \min_w$$

Вместо квадратов складываем модули весов.

Lasso может сделать часть весов ровно нулевыми.

Какие именно веса обнулятся, зависит от данных и силы штрафа λ .

Почему Lasso зануляет веса

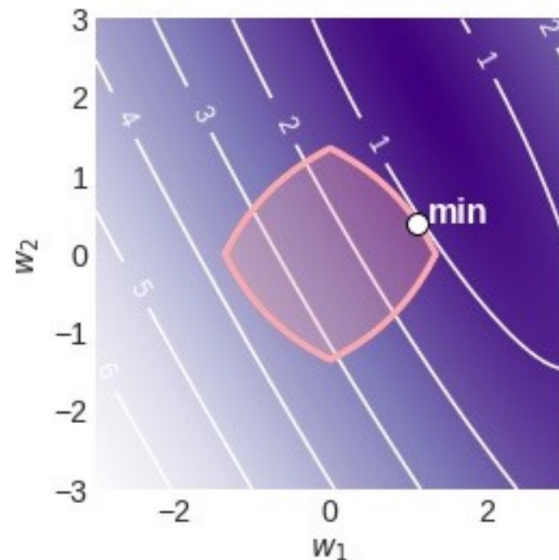


$$|w_1| + |w_2| \leq s$$

Линия уровня ошибки может
коснуться ромба в его вершине.
В этой точке один из весов равен нулю.

Elastic Net объединяет два штрафа

$$J(w) = Q(w) + \lambda_1 \sum_{j=1}^d |w_j| + \lambda_2 \sum_{j=1}^d w_j^2$$



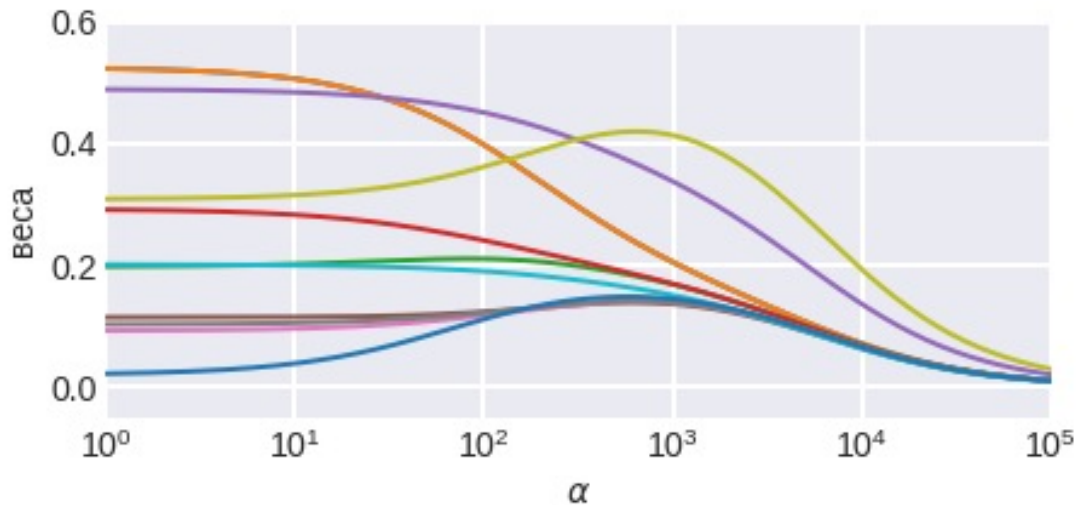
L1-часть позволяет занулять веса.
L2-часть сжимает веса и помогает
при сильно связанных признаках.

λ_1 и λ_2 выбираем по валидации.

Как сила регуляризации влияет на веса

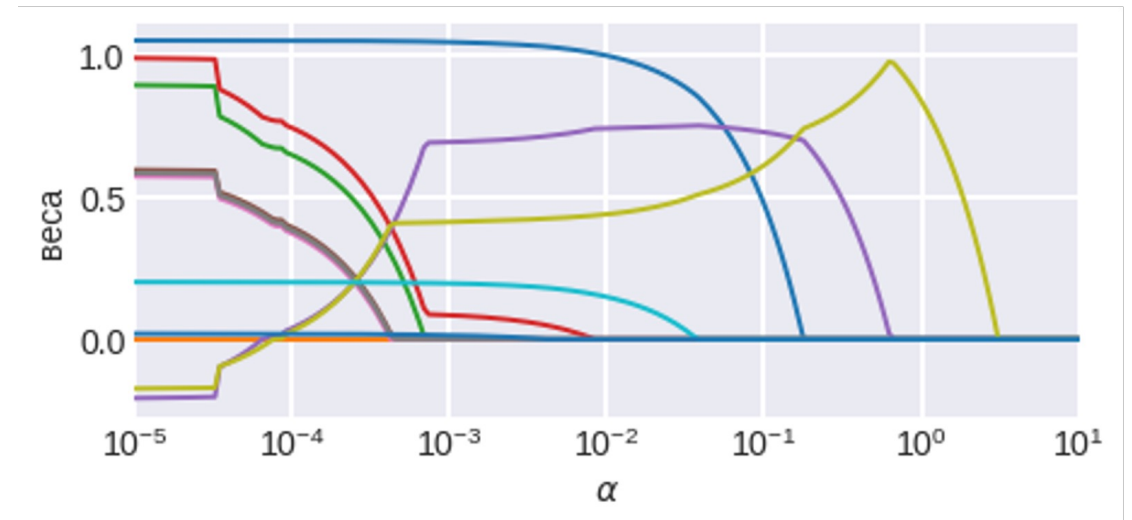
Ridge

Каждая линия показывает один вес. Вправо растёт сила штрафа α .



Веса сжимаются к нулю.

Lasso



Часть весов обнуляется.



Градиентный спуск

03

Направление уменьшения ошибки

Пример на пяти доставках

Длина шага и масштаб признаков

Аналитическое решение: что мешает его использовать

$$w^* = (A^T A)^{-1} A^T y$$

А содержит признаки объектов. В прошлой лекции мы получили эту формулу для весов МНК.

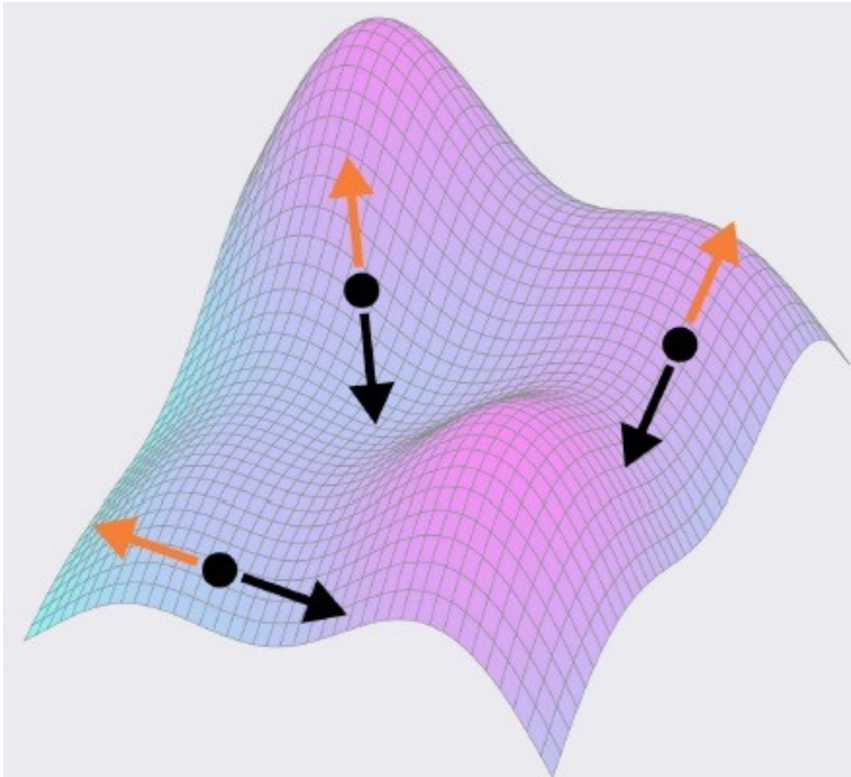
1 Много признаков

Матрица $A^T A$ растёт. На её хранение и решение системы нужны память и время.

2 Зависимые признаки

Если один столбец выражается через другие, обратной матрицы нет.

Градиент показывает направление роста



Высота поверхности показывает значение функции ошибки.

Градиент ∇Q направлен в сторону быстрого роста. На рисунке это оранжевые стрелки.

Чтобы уменьшать ошибку, делаем шаг против градиента.

Производные по всем весам

Градиент объединяет производные по отдельным весам в один вектор.

$$\nabla Q(w) = \left(\frac{\partial Q}{\partial w_0}, \dots, \frac{\partial Q}{\partial w_d} \right)^T$$

Для квадратичной ошибки производную уже нашли в прошлой лекции:

$$\nabla Q(w) = \frac{2}{\ell} A^T (Aw - y)$$

Шаг в направлении антиградиента

Пусть J обозначает критерий Q или $Q + \lambda R$.

В каждой новой точке снова вычисляем градиент.

$$w^{(k+1)} = w^{(k)} - \eta_k \nabla J(w^{(k)})$$

$\eta_k > 0$ задаёт длину шага, или learning rate.

При достаточно малом шаге значение J уменьшается.



Один вес на знакомой выборке

Возьмём те же пять доставок и центрируем расстояние: $z = x - 3$.
Зафиксируем среднее время 25 и будем обучать только наклон w .

z, км	-2	-1	0	1	2
y, мин	18	20	25	29	33

$$a_w(x) = 25 + w(x - 3)$$

$$Q(w) = 0.38 + 2(w - 3.9)^2, Q'(w) = 4(w - 3.9)$$



Первые шаги градиентного спуска

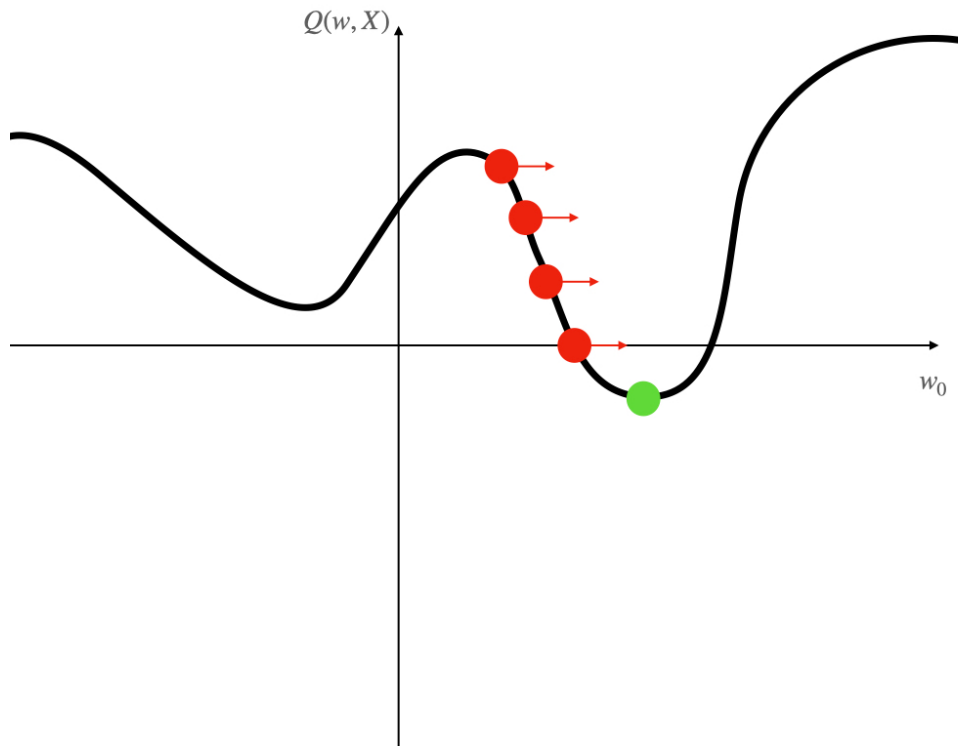
Положим $w^{(0)} = 0$ и $\eta = 0,1$. Первый градиент равен $-15,6$.

Поэтому $w^{(1)} = 0 - 0,1 \cdot (-15,6) = 1,56$.

Оптимум: $w^* = 3,9$ и $Q(w^*) = 0,38$.

Итерация k	Наклон $w^{(k)}$	Градиент Q'	MSE, мин ²
0	0,000	-15,600	30,800
1	1,560	-9,360	11,331
2	2,496	-5,616	4,322
3	3,058	-3,370	1,799
4	3,395	-2,022	0,891

Движение к минимуму

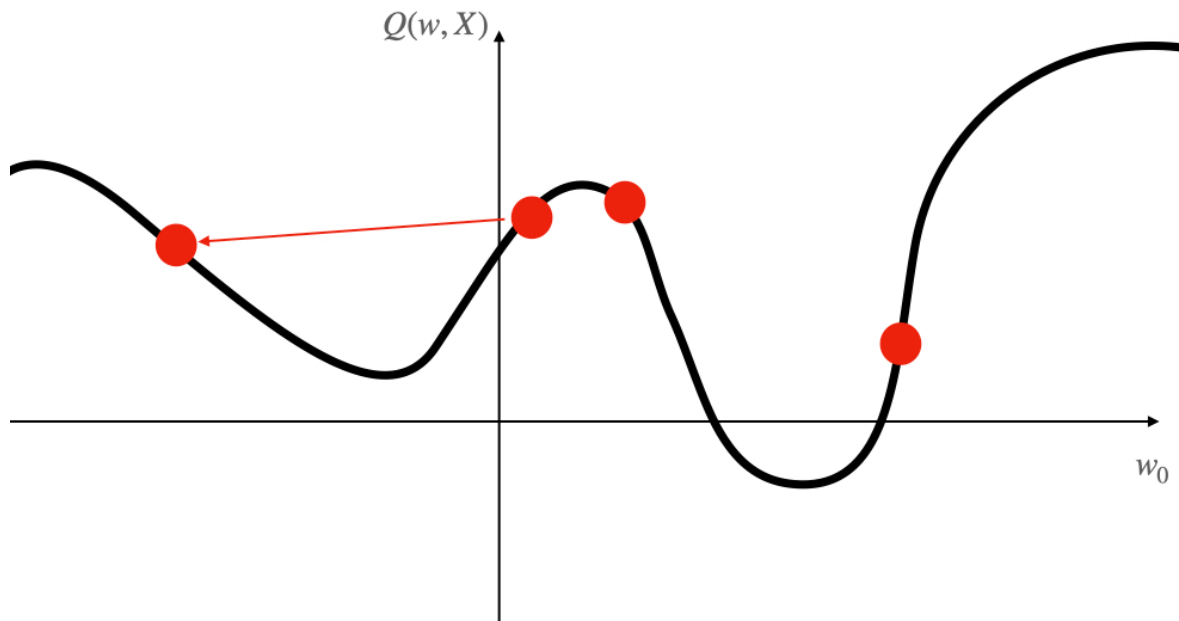


Красные точки показывают
последовательные положения
веса.

Производная отрицательна.
Увеличиваем вес и движемся
к минимуму, отмеченному
зелёным.

Для квадратичной ошибки МНК график по одному весу является параболой.

Слишком большой шаг

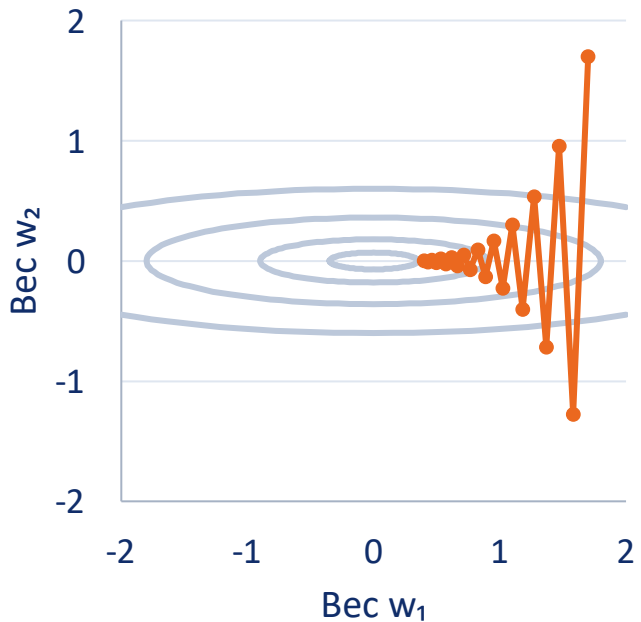


При большом η можно
перескочить через минимум.

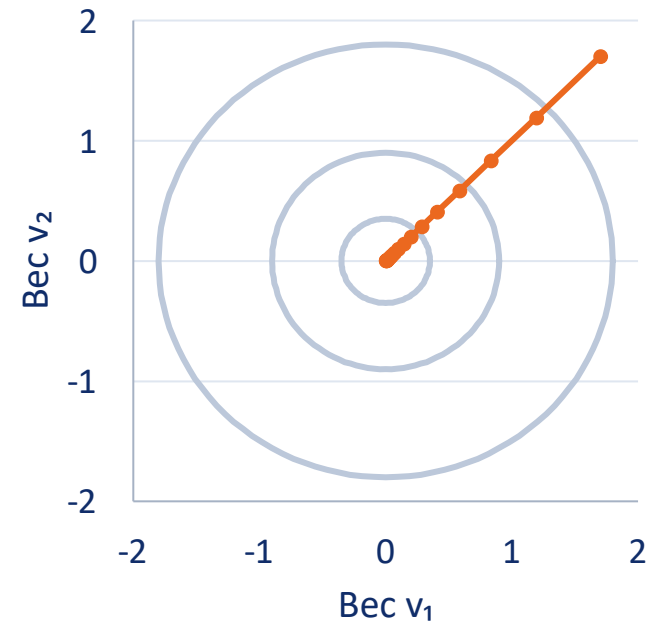
В новой точке производная
меняет знак. Следующий
шаг ведёт в другую сторону.

Масштабирование помогает оптимизации

Разный масштаб



После масштабирования



Слева приходится уменьшать общий шаг из-за узкой «долины».
Справа можно взять больший шаг и быстрее приблизиться к минимуму.



Остановка и проверка сходимости

Критерий остановки	Что проверяем
Малый градиент	Насколько близки к стационарной точке
Малое изменение весов или J	Есть ли заметный прогресс между итерациями
Лимит итераций	Не исчерпан ли вычислительный бюджет
Ранняя остановка по валидации	Не ухудшается ли качество на новых данных

Для МНК нет плохих локальных минимумов: любой минимум даёт наименьшую ошибку.
Если признаки не выражаются друг через друга, оптимальные веса единственны.



SGD и mini-batch

04

Градиент по одному объекту и по порции
Итерации и эпохи
Шум и длина шага

Полный градиент усредняет вклады объектов

$$Q(w) = \frac{1}{\ell} \sum_{i=1}^{\ell} q_i(w), \nabla Q(w) = \frac{1}{\ell} \sum_{i=1}^{\ell} \nabla q_i(w)$$

Для одного веса $q_i(w) = (25 + wz_i - y_i)^2$. При $w = 0$:

Объект i	1	2	3	4	5
Градиент q_i'	-28	-10	0	-8	-32

$$Q'(0) = \frac{-28 - 10 + 0 - 8 - 32}{5} = -15.6$$

SGD: один случайный объект на шаг

Выберем объект i_k случайно и равновероятно. Вместо полного градиента используем производную потери на этом объекте.

$$w^{(k+1)} = w^{(k)} - \eta_k \nabla q_{i_k}(w^{(k)})$$

В нашем примере при $w = 0$ объект 5 даёт градиент -32 .

С шагом $\eta = 0,1$ получаем $w = 3,2$ вместо $1,56$ у полного GD.

Mini-batch: средний градиент по небольшой порции

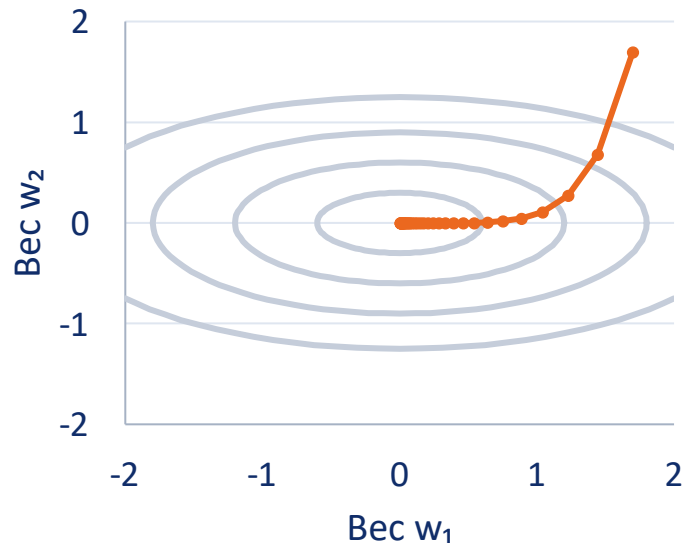
Возьмём случайную порцию B_k из b объектов и усредним их градиенты.

$$g_k = \frac{1}{b} \sum_{i \in B_k} \nabla q_i(w^{(k)}), w^{(k+1)} = w^{(k)} - \eta_k g_k$$

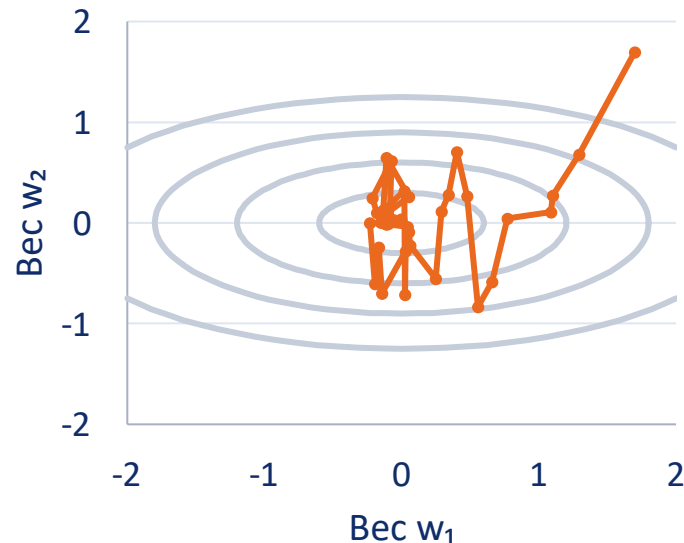
Размер порции	Метод	Что происходит
$b = 1$	SGD	Дешёвый, но шумный шаг
$1 < b < \ell$	Mini-batch GD	Усреднение уменьшает шум оценки
$b = \ell$	Full GD	Точный градиент по всей выборке

Почему траектория SGD шумная

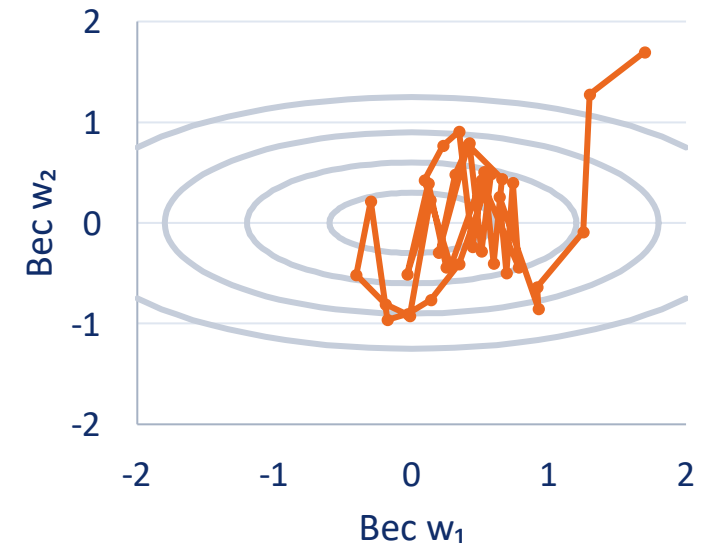
Полный градиент



Mini-batch



SGD



Все три метода уменьшают одну и ту же среднюю ошибку.

У SGD отдельные объекты тянут веса в разные стороны, поэтому шаги шумят.



Длина шага в SGD

В начале большой шаг помогает быстро пройти к хорошим весам.
Рядом с минимумом такой шаг мешает: случайный градиент уводит в сторону.

Поэтому learning rate часто уменьшают в процессе обучения.

$$\eta_k = \frac{\eta_0}{1 + ck}, c > 0$$



Что получили в этой лекции

1 Оценка качества

K-fold даёт несколько оценок качества. Настройки выбираем до финального теста.

2 Управление весами

Ridge сжимает веса, Lasso может их обнулить. Силу штрафа выбираем по валидации.

3 Обучение без обращения матрицы

Градиентный спуск находит веса шаг за шагом по производным ошибки.

4 Более дешёвые шаги

SGD и mini-batch используют часть выборки



Вопросы